

Fondamenti di Informatica

A.A. 2018-2019

Prova intermedia del 5 luglio 2019

NOME _____ COGNOME _____

Durante l'esame non si possono usare appunti né altro materiale.

Esercizio 1

Si dica se ciascuna delle seguenti affermazioni relative all'*information hiding* e ai *tipi di dato astratti* è corretta:

Sì No

- | | | |
|--------------------------|--------------------------|--|
| ☐ | ☐ | Secondo il principio dell' <i>information hiding</i> , i dettagli implementativi vengono tenuti nascosti. |
| ☐ | ☐ | L'insieme dei nomi e delle operazioni disponibili in un tipo di dato astratto è detto classe. |
| ☐ | ☐ | La specifica di un tipo di dato astratto si esprime sempre in modo rigorosamente formale. |
| ☐ | ☐ | Se si applica rigorosamente il principio dell' <i>information hiding</i> , una modifica all'implementazione di una parte di un programma (ad esempio una classe) non ha ripercussioni sul resto del programma. |
| ☐ | ☐ | L' <i>information hiding</i> consente di ottenere programmi più robusti, flessibili e riusabili. |
| ☐ | ☐ | In C++, i tipi di dato astratti si realizzano tipicamente utilizzando le strutture. |
| ☐ | ☐ | Il paradigma data abstraction distingue tra proprietà generali e proprietà specifiche di un tipo. |
| ☐ | ☐ | Definire un nuovo tipo significa definire una collezione di valori e un insieme di operazioni. |
| ☐ | ☐ | In C++, l' <i>information hiding</i> si realizza rendendo pubblici gli attributi di una classe. |
| ☐ | ☐ | In C++ i tipi di dato astratti si realizzano con i meccanismi di incapsulamento offerti dalle classi. |

Esercizio 2

Si scriva la dichiarazione delle seguenti *variabili C++*:

1. Un puntatore a numero intero inizializzato all'indirizzo della variabile *n*: _____
2. Un oggetto di classe *Automobile*: _____
3. Un array di 20 oggetti di classe *Automobile*: _____
4. Un array di 30 puntatori a oggetti di classe *Automobile*: _____
5. Un riferimento a variabile di tipo reale inizializzato alla variabile *z*: _____
6. Un puntatore a funzione che riceva come parametro per valore un numero intero e restituisca come valore di ritorno un numero reale: _____

Esercizio 3

Si dica che cosa stampano a video i seguenti programmi C++, cioè quali nodi compongono la lista *l* e la coda *q* dopo aver eseguito le operazioni sotto riportate:

1 <pre>#include "List.h" int main() { List<int> l; l.insertFront(18); l.insertFront(20); l.insertAfter(l.find(18), 30); l.removeAt(l.find(20)); l.insertBack(50); l.removeFront(); l.print(); return 0; }</pre>	2 <pre>#include "Queue.h" int main() { Queue<char> q; q.insert('a'); q.insert('z'); q.remove(); q.insert('g'); q.insert('p'); q.remove(); q.print(); return 0; }</pre>
--	---

Soluzione:

Esercizio 4

Completare le seguenti affermazioni:

1. In una lista l' _____ di nuovi nodi può avvenire in testa, in coda o in mezzo alla lista.
2. Entrambe le forme (prefissa e postfissa) degli operatori unari _____ possono essere sovraccaricate.
3. In C++, un oggetto allocato _____ deve essere esplicitamente deallocato con l'operatore delete.
4. Quando un puntatore punta a un'altra variabile di tipo puntatore si ha un _____.

Gli esercizi 5 e 6, qui sotto, dovranno essere svolti al calcolatore e consegnati secondo le seguenti modalità:

- **Compilare il modulo che compare nella finestra che avete trovata aperta sul vostro PC. Se non la trovate, fate doppio click sull'icona Esame che compare sul desktop.**
 - **Creare un file .zip contenente il codice sorgente (cioè i file .cpp), caricarlo e inviarlo.**
-

Esercizio 5

La *mappa logistica* fu proposta nel 1845 da P.F. Verhulst come modello matematico per lo studio dei fenomeni di crescita delle popolazioni biologiche. Se indichiamo con x_n la densità di una popolazione alla generazione n – densità normalizzata tra 0 (popolazione assente) e 1 (densità massima) – e con $\lambda \in \mathbf{R}$ il tasso di crescita costante della popolazione stessa da una generazione all'altra, allora la densità x_n della popolazione alla generazione n è data da:

$$\begin{cases} x_n = \lambda x_{n-1}(1 - x_{n-1}) \\ x_0 = \frac{1}{5} \end{cases}$$

Si scriva in C++ la funzione ricorsiva *mappa_logistica* che riceva in ingresso (ovvero come parametri) i valori di n (un numero intero) e di λ (un numero reale), calcoli e restituisca come valore di ritorno la densità x_n della popolazione alla n -esima generazione (un numero reale). Nel caso in cui n assuma un valore negativo, la funzione restituisce 0.

Si scriva quindi un programma C++ che operi come segue: definisca un array *lambda* di 7 numeri reali e lo inizializzi con i valori seguenti {1, 2, 3, 3.25, 3.5, 3.75, 4}; per ciascuno dei valori di λ contenuti nell'array *lambda*, chiami la funzione *mappa_logistica* per tutti i valori di n compresi tra 0 e 10, stampandone a video il valore di ritorno.

Esercizio 6

Per gestire calcoli con polinomi di una sola variabile a coefficienti interi e reali, si sviluppi in C++ il template di classe *Polinomio* avente i seguenti attributi: il grado *_n* del polinomio (un numero intero), il puntatore *_p* a un array di $n + 1$ oggetti di tipo T che rappresentano i coefficienti dei termini del polinomio. Si implementino, inoltre, i seguenti metodi:

- Il costruttore di default che inizializzi *_n* a zero e *_p* a NULL.
- Un costruttore con parametri che riceva in ingresso (ovvero come parametri) il grado g di un polinomio (un numero intero) e un array *c* di $g + 1$ oggetti di tipo T. Il costruttore opera come segue: alloca un array di $g + 1$ oggetti di tipo T e ne assegna il puntatore a *_p*, inizializza quindi *_n* con il valore di g e l'array puntato da *_p* copiando ciascun elemento dell'array *c* nel corrispondente elemento dell'array puntato da *_p*. Se g assume un valore negativo o l'allocazione dinamica non ha successo, il costruttore inizializza *_n* a zero e *_p* a NULL.
- Il costruttore di copia.
- Il distruttore.
- I selettori (un metodo per ciascun attributo).
- L'operatore `==` per il quale due polinomi sono uguali se hanno lo stesso grado e gli stessi coefficienti.
- L'operatore di assegnamento.
- Il metodo *calcola* che riceva in ingresso (ovvero come parametro) un oggetto *x* di tipo T, calcoli e restituisca come valore di ritorno il valore y (un oggetto di tipo T) dato da: $y = a_n x^n + a_{n-1} x^{n-1} + \dots + a_2 x^2 + a_1 x + a_0$, essendo gli a_i i coefficienti dei termini del polinomio. *Nota*: per le potenze si può usare la funzione *pow* disponibile in *cmath*.

Si sviluppi quindi la funzione *main* che operi come segue:

- Dichiarare un array *coeff* di tre numeri reali e inizializzarli tutti i suoi elementi a zero.
- Dichiarare un oggetto *prev_pol* di tipo *Polinomio*, istanziato con il tipo concreto *double*, utilizzando il costruttore con parametri al quale viene passato l'array *coeff*.
- Finché l'utente lo desidera operi come segue: chieda all'utente di inserire da tastiera i tre coefficienti (tre numeri reali) di un polinomio di secondo grado e li copi nei corrispondenti elementi dell'array *coeff*; crei un oggetto *pol* di tipo *Polinomio*, istanziato con il tipo concreto *double*, utilizzando il costruttore con parametri al quale è passato l'array *coeff*; verifichi se il polinomio appena inserito dall'utente è uguale al polinomio rappresentato dall'oggetto *prev_pol* e in caso affermativo stampi a video un messaggio per l'utente; invochi su *pol* il metodo *calcola* passando un valore di x inserito dall'utente e stampi a video il risultato; assegni l'oggetto *pol* all'oggetto *prev_pol*.

Soluzione

Esercizio 1 – Soluzione (1.5 punti, 0.15 per ogni risposta esatta)

Sì, No, No, Sì, Sì, No, No, Sì, No, Sì.

Esercizio 2– Soluzione (1.5 punto, 0.25 per ogni risposta esatta)

```
int* p = &n; Automobile a; Automobile a[20]; Automobile* a[30]; double& r = z; double (*pf)(int);
```

Esercizio 3 – Soluzione (1.0 punto, 0.5 per ogni risposta esatta)

1. 30 50; 2. p g

Esercizio 4 – Soluzione (1.0 punto, 0.25 per ogni risposta esatta)

Inserimento, ++ e --, dinamicamente, puntatore a puntatore.

Esercizio 5 – Soluzione (2.0 punti, 1.0 – 1.0)

```
#include <iostream>
using namespace std;

double mappa_logistica(int n, double lambda) {
    if (n < 0) return 0.0;
    if (n == 0)
        return 1.0 / 5.0;
    else {
        double x = mappa_logistica(n - 1, lambda);
        return lambda * x * (1 - x);
    }
}

int main() {
    const int dim = 7;
    double lambda[dim] =
        { 1.0, 2.0, 3.0, 3.25, 3.5, 3.75, 4.0 };
    for (int i = 0; i < dim; i++) {
        cout << "Lambda = " << lambda[i] << endl;
        for (int j = 0; j < 10; j++)
            cout<<mappa_logistica(j, lambda[i])<<" ";
        cout << endl; }
    return 0;
}
```

Esercizio 6 – Soluzione (4.0 punti, 0.6 – 0.2 – 0.4 – 0.4 – 0.2 – 0.2 – 0.2 – 0.4 – 0.4 – 0.4 – 0.6)

```
#include <iostream>
#include <cmath>
using namespace std;

template <typename T>
class Polinomio {
public:
    Polinomio();
    Polinomio(int g, T c[]);
    Polinomio(const Polinomio<T>& p);
    ~Polinomio();
    int getGrado() const;
    const T* getCoefficienti() const;
    int operator==(const Polinomio<T>& p) const;
    const Polinomio<T>& operator=(const Polinomio<T>& s);
    double calcola(T x) const;
private:
    int _n; T* _p; };

template <typename T>
Polinomio<T>::Polinomio() {_n = 0; _p = NULL;}

template <typename T>
Polinomio<T>::Polinomio(int g, T c[]) {
    if (g > 0) {
        _p = new T[g + 1];
        if (_p != NULL) {
            _n = g;
            for (int i = 0; i < g + 1; i++)
                _p[i] = c[i];
            return; }
        _n = 0;
        _p = NULL; }

template <typename T>
Polinomio<T>::Polinomio(const Polinomio<T>& p) {
    if (p._n > 0) {
        _p = new T[p._n + 1];
        if (_p != NULL) {
            _n = p._n;
            for (int i = 0; i < p._n + 1; i++)
                _p[i] = p._p[i];
            return; }
        _n = 0;
        _p = NULL; }

template <typename T>
Polinomio<T>::~~Polinomio() {
    if (_p != NULL) delete[] _p; }

template <typename T>
int Polinomio<T>::getGrado() const {
    return _n; }

template <typename T>
const T* Polinomio<T>::getCoefficienti() const {
    return _p; }

template <typename T>
int Polinomio<T>::operator==(const Polinomio<T>& p) const {
    for (int i = 0; i < _n + 1; i++)
        if (_p[i] != p._p[i]) return 0;
    return 1; }

template <typename T>
const Polinomio<T>& Polinomio<T>::operator=
(const Polinomio<T>& p) {
    if (this != &p) {
        if (_p != NULL) delete[] _p;
        if (p._n > 0) {
            _p = new T[p._n + 1];
            if (_p != NULL) {
                _n = p._n;
                for (int i = 0; i < p._n + 1; i++)
                    _p[i] = p._p[i];
                return *this; } }
        _n = 0;
        _p = NULL; }
    return *this; }

template <typename T>
double Polinomio<T>::calcola(T x) const {
    double y = _p[0];
    for (int i = 1; i < _n + 1; i++)
        y += _p[i] * pow(x, i);
    return y; }

int main() {
    const int dim = 3;
    double coeff[dim] = { 0 };
    Polinomio<double> prev_pol(dim - 1, coeff);
    char c = 'n';
    do {
        cout << "Inserire coefficienti: " << endl;
        for (int i = 0; i < dim; i++) {
            cout << "Coefficiente " << i << ": ";
            cin >> coeff[i]; }
        Polinomio<double> pol(dim - 1, coeff);
        if (pol == prev_pol)
            cout << "Stesso polinomio!" << endl;
        cout << "Inserire un valore per x: ";
        double x = 0; cin >> x;
        cout << "y = " << pol.calcola(x) << endl;
        prev_pol = pol;
        cout << "Vuoi continuare (s/n)? " << endl;
        cin >> c;
    } while (c != 'n');
    return 0; }
```